

A Comprehensive Analysis of the AKMA+ Protocol

Yueming Li^{1,2}, Long Chen^{1,✉}, and Zhenfeng Zhang^{1,✉}

¹ Institute of Software, Chinese Academy of Sciences, Beijing, China

² University of Chinese Academy of Sciences

{yueming2021, chenlong, zhenfeng}@iscas.ac.cn

Abstract. With the rapid advancement of 5G networks and the increasing demand for secure application access, the Authentication and Key Management for Applications (AKMA) framework was developed by the 3rd Generation Partnership Project (3GPP) to provide unified authentication and key management for diverse 5G services. In response to the security and privacy concerns identified in the current AKMA protocol, as outlined in 3GPP TR 33.835, Yang et al. proposed an enhanced, standard-compatible 5G AKMA protocol known as AKMA+ [14]. This paper presents a comprehensive analysis of AKMA+, discovering two critical vulnerabilities: (1) the compromise of the AKMA Anchor Function (AAnF), which enables adversaries to impersonate legitimate users; and (2) the persistent storage of multiple anchor keys, which heightens the risk of key exposure. These vulnerabilities arise from the reliance on the authentication framework inherent in existing AKMA+ models. This architectural dependency introduces fundamental security risks that cannot be adequately mitigated through incremental modifications to the current design.

Furthermore, we observe that AKMA+ faces challenges in aligning with the standard account-based authentication model, which is incompatible with existing user practices within information systems. Additionally, we find that providing account-based authentication functionality without compromising privacy poses significant difficulties³.

Keywords: 5G Authentication, Key Management, Security Vulnerability, Privacy Protection

1 Introduction

The fifth generation (5G) of mobile networks represents a transformative leap in communication technology, offering unprecedented bandwidth, ultra-low latency, and massive connectivity for diverse applications ranging from enhanced mobile broadband to mission-critical services and massive Internet of Things (IoT) deployments [1, 8–10]. With this expanded scope, 5G networks face heightened security and privacy challenges, particularly in authenticating and securing communications between User Equipment (UE) and various Application Functions (AFs).

To address these challenges, the 3rd Generation Partnership Project (3GPP) developed the Authentication and Key Management for Applications (AKMA) framework,

³ This work is supported by National Key R&D Program of China 2022YFB3102500 and CAS Project for Young Scientists in Basic Research, Grant No.YSBR-035

specified in Technical Specification (TS) 33.535 [5]. AKMA provides a standardized method for secure key management across diverse applications in 5G networks, leveraging the existing 5G authentication infrastructure to derive application-specific keys. This approach aims to enhance security, privacy, interoperability, and scalability in 5G communication, making it a fundamental building block for the 5G ecosystem. Leading vendors like Ericsson, Nokia, and Huawei, along with major operators such as AT&T and Verizon, are adopting the AKMA protocol for their 5G networks.

The 3GPP Technical Report (TR) 33.835 [2] lists seventeen key issues for AKMA in 5G networks, among which five are protocol-level security and privacy challenges: (1) *Mutual authentication between UE and the AKMA anchor function (AAnF)* (KI#3); (2) *User privacy*, preventing Subscription Permanent Identifier (SUPI) exposure (KI#5); (3) *Secure communication between UE and AF*, ensuring session key confidentiality (KI#6); (4) *Protection of subscriber's personal information in both control and data traffic* (KI#7); and (5) *Application key freshness* for forward secrecy (KI#16). The other issues are related to architecture, interfaces, APIs, or regulatory compliance, and are not solvable at the protocol level. Similarly, Yang et al. also identified several vulnerabilities in the original AKMA protocol that could lead to security and privacy breaches [13].

While several related works and candidate solutions have been proposed to enhance the security and privacy of AKMA, none of them systematically address all five of the aforementioned key issues. For example, Akman et al. [6] and Khan et al. [11] only partially address mutual authentication or key freshness, but fail to ensure user privacy, secure UE-AF communication, or protection of personal information in both control and data traffic. Other candidate solutions listed in TR 33.835 [2] either lack formal verification, do not maintain standard compatibility, or only address isolated issues.

Published in USENIX 2025, Yang et al. [14] conducted a detailed analysis of AKMA and identified multiple security and privacy weaknesses. Their work pointed out that the AAnF could be malicious, potentially accessing sensitive data transmitted between the UE and AF, or injecting malicious payloads. Under this security assumption, their research led to the development of AKMA+, an enhanced protocol designed to address these vulnerabilities while maintaining compatibility with existing standards. AKMA+ introduced several key improvements, including countermeasures to protect communication between UEs and AFs from attackers (even those within the home network), ensuring mutual authentication between the UE and the AAnF, and preventing linkability attacks by obfuscating subscriber and key identifiers.

While AKMA+ claims to represent a significant advancement over the original AKMA protocol, our research reveals that under the malicious AAnF assumption proposed by them, the AKMA+ scheme still contains critical vulnerabilities that undermine its security and privacy guarantees. AKMA+ assumes the user's long-term key K is secure since it resides in the SIM card (trusted hardware), whereas the other AKMA+ keys lack such SIM-based secure storage. In this paper, we present a comprehensive analysis of AKMA+, identifying three major vulnerabilities:

1. **AAnF Compromise Attack:** Our analysis demonstrates that if the adversary, as posited in AKMA+, is capable of compromising AAnF [14], they would be able to impersonate legitimate UEs to AFs, which is a consequence of the centralized trust architecture of AKMA+.

2. **UE Compromise Attack:** Unlike the original AKMA protocol, which used ephemeral keys, AKMA+ requires long-term storage of multiple AKMA anchor keys, which can be stolen by an adversary due to the lack of hardware protection on the SIM card [14]. We show how this architectural change extends the exposure window for critical cryptographic materials and amplifies the impact of potential UE compromises.
3. **Ambiguity in Account Identity:** The AKMA+ architecture lacks a clear definition for account names, which prevents it from effectively serving users who interact with systems via accounts. This is a critical gap, as an AF needs to distinguish between users to provide personalized services, a requirement also highlighted in TR 33.835 [3]. Although adding account names seems like a straightforward solution, doing so would create new attack vectors and privacy concerns. Our analysis indicates that the current protocol architecture cannot feasibly accommodate an account name field without a major redesign.

Our analysis reveals that these vulnerabilities are not merely implementation flaws but stem from fundamental architectural limitations in the AKMA+ protocol. The centralized trust model, with the AAnF as the primary trust anchor, creates inherent security and privacy weaknesses that cannot be fully addressed within the current architectural framework. This finding has significant implications for the security architecture of 5G networks, suggesting the need for a more distributed trust model that can withstand the compromise of individual components. Specifically, our contributions are threefold:

- We provide a detailed security analysis of the AKMA+ protocol, identifying previously unknown vulnerabilities that compromise its security and privacy guarantees.
- We demonstrate that these vulnerabilities arise from fundamental architectural limitations in AKMA+, particularly its reliance on a centralized trust model.
- We propose several directions for future protocol improvements to address these vulnerabilities, including the adoption of hardware-based secure storage and modifications to the protocol architecture.

Technique Overview

The AKMA+ protocol introduces specific security assumptions for the AAnF and UE to address potential vulnerabilities [14]. The AAnF is assumed to be a "covert entity" that adheres to prescribed 5G network protocols, such as providing authentic and executing authentication procedures correctly. However, it may access sensitive data transmitted between the UE and AF, inject malicious payloads, posing risks to users. Based on this assumption, we point out three attacks that currently exist on AKMA+.

- **AAnF Compromise Attack:** Compared to AKMA, AKMA+ adopts a more complex key management mechanism by generating multiple key identifier pairs and introducing DH key exchange to enhance the privacy of session keys. However, it still relies on AAnF as a centralized trust anchor. If an attacker compromises AAnF, the attacker can access all key identifier pairs stored in AAnF. Although DH key exchange is used, its temporary nature means it does not provide confidentiality. Consequently, an adversary controlling AAnF can impersonate the UE to access the AF.
- **UE Compromise Attack:** AKMA+ replaces the temporary key mechanism in AKMA with the long-term storage of multiple anchor key-identity pairs to preserve user identity privacy. However, the long-term storage of multiple anchor keys significantly

increases the risk of key exposure. Suppose the UE is compromised (e.g., through device theft or malware infection). In that case, an attacker can obtain the anchor keys and their associated identifiers stored on the UE, enabling them to impersonate a legitimate user when communicating with the AF. Since the architectural design of AKMA+ lacks mandatory hardware-based secure storage mechanisms (such as USIM-based storage), adversaries can easily gain access to this sensitive information.

- **Ambiguity in Account Identity:** AKMA+ enhances user privacy by preventing the exposure of UE identity information to the AF. In this design, the AF requests the session key from AAnF using the key identifier provided by the UE. However, this approach results in the AF being unable to identify the specific identity of the user or provide personalized services. Consequently, the AF cannot differentiate between users to offer customized functionalities or services, such as account management or personalized content recommendations. While we propose four possible methods for incorporating account names under the condition of standard compatibility, analysis indicates that these approaches may introduce new privacy concerns.

2 AKMA Protocol

2.1 Architecture of AKMA

The AKMA architecture [5], illustrated in Figure 1, consists of several key network elements that interact to provide secure authentication and key management for applications in 5G.

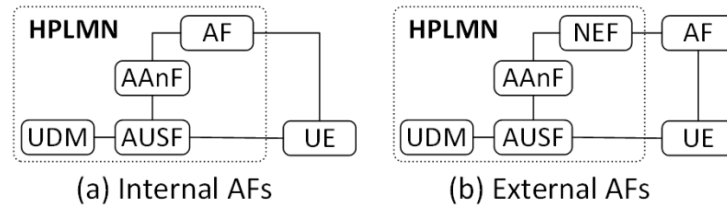


Fig. 1. System Architecture of AKMA [14]

The main components of the AKMA architecture are:

- **User Equipment (UE):** Represents a subscriber of the AKMA protocol, comprising Mobile Equipment (ME) and a Universal Integrated Circuit Card (UICC). A Universal Subscriber Identification Module (USIM) resides in the UICC. The ME can be any mobile device, such as a smartphone or IoT device, capable of incorporating the UICC.
- **Home Public Land Mobile Network (HPLMN):** The network operated by the user's home service provider, where the user's subscription information and authentication credentials are stored and managed. The HPLMN contains several functional entities:

- **United Data Management (UDM)**: Stores AKMA subscription data and authentication vectors.
 - **Authentication Server Function (AUSF)**: Generates and manages authentication keys, which are used to derive subsequent keys for protecting communication between UE and AFs.
 - **AKMA Anchor Function (AAnF)**: Stores authentication keys and anchor keys for the AKMA protocol, and generates application keys for AFs.
 - **Network Exposure Function (NEF)**: Authorizes external AFs to access the AKMA service and forwards AF requests to the AAnF.
- **Application Function (AF)**: The application provider refers to any service offering applications that users wish to access, with AFs potentially residing within or outside the HPLMN.

2.2 AKMA Key Hierarchy

As shown in Figure 2, the AKMA protocol uses a hierarchical key structure to secure UE-AF communications.

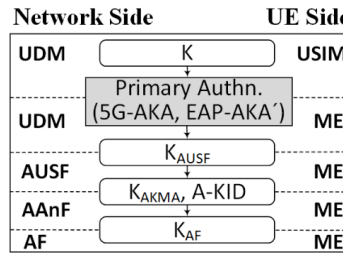


Fig. 2. AKMA Key Hierarchy [14]

The key hierarchy in AKMA includes following elements:

- **Long-term Key (K)**: Long-term Key is the pre-shared key between UE and UDM, used as the root of trust for all subsequent key derivations. This key is securely stored in the USIM and the UDM.
- **Authentication Key (K_{AUSF})**: Derived by UDM and UE from K during primary authentication. This key serves as the foundation for generating further keys to secure communication sessions and services.
- **AKMA Anchor Key (K_{AKMA})**: Derived by AUSF and UE from K_{AUSF} using a key derivation function (KDF). This key is specific to the AKMA framework and serves as the basis for deriving application-specific keys.
- **AKMA Key Identifier (A-KID)**: A globally unique identifier for K_{AKMA}, used to reference the AKMA anchor key in subsequent protocol steps. The A-KID is in Network Access Identifier (NAI) format, consisting of a username part (Routing Indicator and AKMA Temporary UE Identifier) and a realm part (HPLMN Identifier).

- **AKMA Application Key (K_{AF})**: Derived by AAnF and UE from K_{AKMA} , using the AF's identity (AF_ID) as input to the KDF. This key is used to secure communication between the UE and the AF.

2.3 AKMA Protocol Procedures

The AKMA protocol [4] operates in two main phases: (1) deriving AKMA material after primary authentication, and (2) deriving the AKMA application key for communication with an AF. We describe these procedures in detail below.

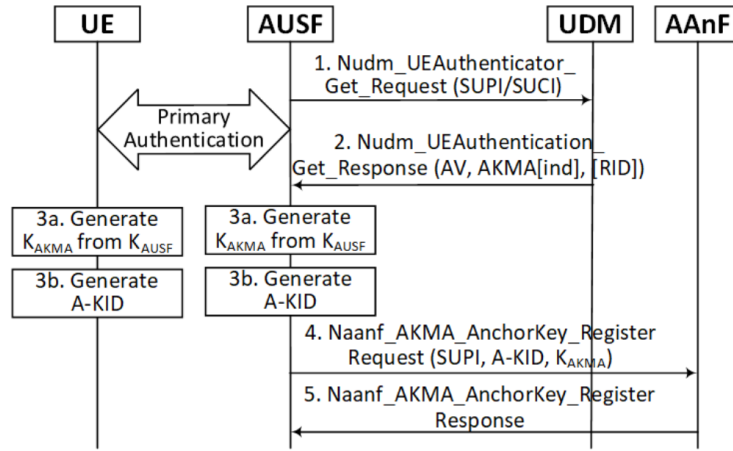


Fig. 3. Deriving AKMA Material After Primary Authentication [14]

Primary Authentication and Deriving AKMA Material. Deriving AKMA material after primary authentication is illustrated in Figure 3 and consists of the following steps:

The process begins when the UE initiates primary authentication with its HN using either the 5G Authentication and Key Agreement (5G-AKA) or the Extensible Authentication Protocol for Authentication and Key Agreement Prime (EAP-AKA'). During this phase, the AUSF sends a request to the UDM containing the user's identity (SUPI or SUCI). In response, the UDM provides a 5G authentication vector (AV)—which includes RAND, AUTN, XRES*, and K_{SEAF} (serving network authentication key)—along with an AKMA indication and a Routing Indicator (RID).

Following a successful primary authentication, both the UE and the AUSF independently derive the AKMA anchor key (K_{AKMA}) from a session key (K_{AUSF}) using a standardized Key Derivation Function. Concurrently, they both generate a unique AKMA Key Identifier (A-KID) in NAI format, constructed by combining the RID, an AKMA Temporary UE Identifier (A-TID), and the home network's realm identifier. As the final step, the AUSF securely transfers the derived K_{AKMA} , the A-KID, and the user's SUPI to the AAnF, which then stores these credentials for future use.

Deriving AKMA Application Key for AF.

Before communication between the UE and AF begins, an AKMA application key (K_{AF}) must be derived to secure the session. Figure 4 illustrates this process for the scenario where the AF resides in the HPLMN.

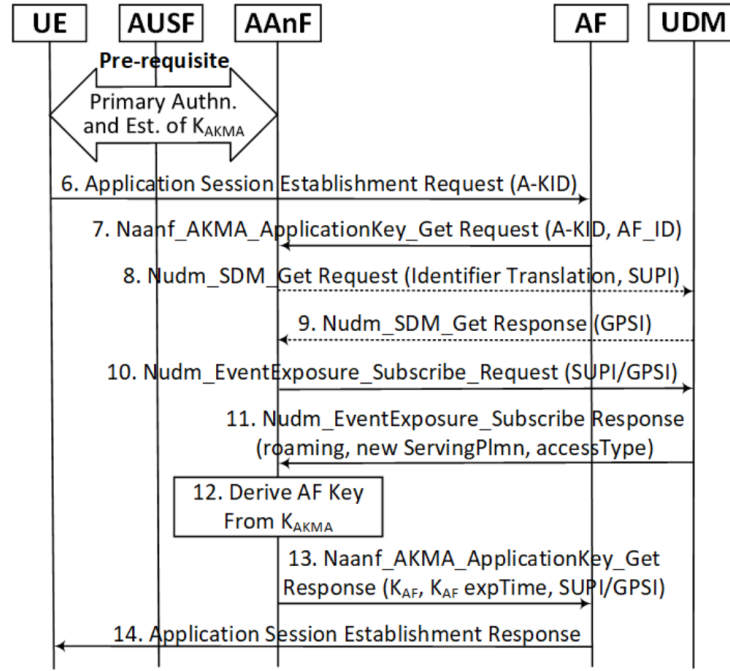


Fig. 4. Deriving K_{AF} for AF located within HPLMN

Following the initial key establishment, the application access phase begins when the UE sends a request to an AF, presenting its A-KID. The AF uses this A-KID to locate the user's HPLMN and forwards it, along with its own identifier (AF_ID), to the AAnF to request an application-specific key.

Upon receiving the request, the AAnF may perform several preparatory actions: it can optionally query the UDM to retrieve the user's GPSI if required by the AF, and it also checks the UE's roaming status with the UDM. Subsequently, the AAnF derives the K_{AF} using the K_{AKMA} and the AF's identifier. Critically, the UE performs the exact same key derivation independently using the same inputs. The AAnF then delivers the derived K_{AF} , its expiration time, and the user's identity (SUPI or GPSI) to the AF. With both the UE and AF now possessing the shared secret K_{AF} , they can proceed to establish a secure communication channel, for instance, by using it as a pre-shared key in a TLS handshake.

In situations where the AF is outside the HPLMN, the process remains comparable, with the inclusion of the NEF serving as an intermediary between the AF and AAnF.

NEF relays messages between AF and AAnF, providing an additional layer of security and access control for external AFs.

2.4 Security and Privacy Considerations in AKMA

The AKMA protocol provides a standardized framework for secure key management in 5G networks, but several security and privacy vulnerabilities have been identified in the 3GPP TR 33.835 [3]. These vulnerabilities highlight the need to enhance the AKMA protocol while ensuring compatibility with existing 5G standards. The identified vulnerabilities are categorized into key issues, with five specific issues being particularly significant from a protocol perspective.

1. **Key Issue #3: Mutual Authentication between UE and AAnF:** The AKMA protocol lacks direct mutual authentication between UE and AAnF, creating potential security risks where unauthorized UEs may interact with AAnF or fake AAnFs may communicate with UEs.
2. **Key Issue #5: User Privacy:** The AKMA protocol exposes identifiers that can be used to track users across different applications, compromising user privacy.
3. **Key Issue #6: Secure Communication between UE and AF:** The session keys used in AKMA are vulnerable to interception by the AAnF, reuse across different sessions, and derivation from compromised long-term keys.
4. **Key Issue #7: Protecting Subscriber's Personal Information:** The AKMA protocol does not adequately protect subscriber's A-KID in control and data traffic.
5. **Key Issue #16: Application Key Freshness:** The AKMA protocol does not ensure the freshness of application keys, potentially allowing replay attacks.

3 The AKMA+ Protocol

The AKMA+ protocol, proposed by Yang et al. [14], aims to address the five issues mentioned earlier by introducing improvements in security and privacy. This section provides a detailed explanation of the AKMA+ protocol, including its design principles, procedural steps, and key upgrades that make it different from the original AKMA protocol.

3.1 Design Principles of AKMA+

The design of AKMA+ is guided by four key security goals:

1. **UE Indistinguishability:** Prevent attackers from distinguishing which UE among multiple UEs has accessed an AF, thereby protecting user privacy and safeguarding subscribers' personal information.
2. **Secrecy of Session Key:** Establish a secret session key between UE and AF while ensuring the confidentiality of the key exchange process, thereby preserving secure and reliable communication.
3. **Forward Secrecy:** Ensure that even if long-term secret keys are compromised, the security of previous session keys remains, maintaining the confidentiality of past communication sessions.

4. **Mutual Authentication between UE and AAnF:** Enable UE and AAnF to mutually authenticate each other, verifying the legitimacy of their identities and preventing impersonation attacks.

To achieve these goals, AKMA+ introduces several key enhancements to the original AKMA protocol:

- **Multiple Key-Identifier Pairs:** Instead of generating a single AKMA anchor key and identifier, AKMA+ generates multiple key-identifier pairs for each UE, which are then shuffled among users to prevent AAnF linkability attacks.
- **Privacy-Hardened Application Key:** AKMA+ uses Diffie-Hellman key exchange to generate a secure application key, ensuring UE-AF communication remains confidential and inaccessible to the AAnF.
- **Challenge-Response Mechanism:** AKMA+ implements a challenge-response mechanism between the UE and AAnF to achieve mutual authentication.

3.2 AKMA+ Protocol Procedures

The AKMA+ protocol maintains the same overall structure as AKMA but introduces significant changes to the internal processing of messages, which are detailed below to emphasize the key differences from the original. As illustrated in Figure 5 and Figure 6, these modifications are color-coded by their function: the red changes achieve user unlinkability for privacy, while the blue ones establish mutual authentication and forward secrecy for security.

Deriving AKMA Anchor Keys. Figure 5 illustrates the procedure for deriving AKMA anchor keys in AKMA+. The procedure consists of the following steps:

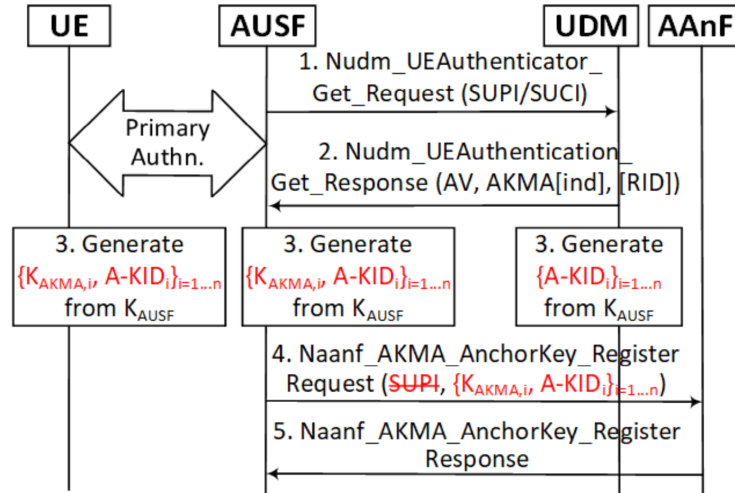


Fig. 5. AKMA+: Deriving AKMA Anchor Keys [14]

1. 2) **Primary Authentication:** Consistent with the original AKMA, the UE undergoes primary authentication with the HPLMN, establishing K_{AUSF} , and AUSF requests the 5G authentication vector (AV).
3. **Multiple Key-Identifier Pairs Generation:** Unlike AKMA, which generates a single key-identifier pair, to prevent multiple logins of a UE from being linked, AKMA+ generates multiple $\{K_{\text{AKMA},i}, \text{A-KID}_i\}_{i=1,\dots,n}$ key-identifier pairs for each UE. n is the number of key-identifier pairs derived during each AKMA key generation following primary authentication. The generation of each key-identifier pair follows the 3GPP specifications with additional parameters to ensure uniqueness and freshness, $K_{\text{AKMA},i} = \text{KDF}(K_{\text{AUSF}}, \text{FC} || P_0 || L_0 || P_1 || L_1 || P_2 || L_2 || P_3 || L_3)$.
 - $\text{FC} = 0x80$ is a fixed function code
 - $P_0 = \text{"AKMA"}$ and L_0 is its length
 - $P_1 = \text{SUPI}$ and L_1 is its length
 - $P_2 = \text{Counter}$, a sequential value for generating multiple keys, and L_2 is its length
 - $P_3 = \text{Date}$, representing the key generation timestamp, and L_3 is its length
 Similarly, each AKMA Key Identifier (A-KID_{*i*}) is generated in two steps. First, an AKMA Temporary Identifier (A-TID_{*i*}) is derived, $\text{A-TID}_i = \text{KDF}(K_{\text{AUSF}}, \text{FC} || P_0 || L_0 || P_1 || L_1 || P_2 || L_2 || P_3 || L_3)$. $\text{FC} = 0x81$, $P_0 = \text{"A-TID"}$, and the other parameters are the same as above. Second, A-KID_{*i*} is constructed in NAI format $\text{A-KID}_i = \text{RID} || \text{A-TID}_i || \text{"@"} || \text{Realm}$. The identifiers $\{\text{A-KID}_i\}_{i=1,\dots,n}$ are stored by UDM as $(\text{SUPI}, \{\text{A-KID}_i\}_{i=1,\dots,n})$ in its database.
4. 5) **Key-Identifier Pairs Shuffling:** AKMA+ introduces a shuffling mechanism to prevent the AAnF from associating specific key-identifier pairs with particular UEs. Suppose there are k UEs, each generating n key-identifier pairs, resulting in a total of $N = n \cdot k$ pairs. AUSF combines all pairs into a single set $S = \{(K_{\text{AKMA},i}^{(u)}, \text{A-KID}_i^{(u)}) \mid u \in \{1, \dots, k\}, i \in \{1, \dots, n\}\}$. u denotes the UE index, and i represents the pair index for each UE. AUSF then applies a random permutation to shuffle the set S , resulting in $\{(K_{\text{AKMA},j}, \text{A-KID}_j) \mid j \in \{1, \dots, N\}\}$. AUSF transmits the shuffled key-identifier pairs to AAnF, which stores them for future use.

Deriving AKMA Application Key for AF. Figure 6 illustrates the procedure for deriving the AKMA application key for AF in AKMA+. The procedure consists of the following steps:

6. **Application Session Establishment Request:** To ensure session freshness and resist replay attacks, UE generates a random nonce $a_1 \in \{0, 1\}^{128}$ and selects an ephemeral DH secret key $u \in_R \mathbb{Z}_p^*$, calculating $U = uG$, G is the generator of the group $\mathbb{G}$. UE encrypts the pair (U, a_1) using the AF's public key PK_{AF} and a public key encryption algorithm PKEnc , $\text{CT}_{\text{UE} \rightarrow \text{AF}} = \text{PKEnc}_{\text{PK}_{\text{AF}}}(U, a_1)$. For mutual authentication between UE and AAnF, UE generates another random nonce a_2 as a challenge and encrypts the tuple $(\text{AF_ID}, a_2)$ using $K_{\text{AKMA},i}$, $\text{CT}_{\text{UE} \rightarrow \text{AAnF}} = \text{AEnc}_{K_{\text{AKMA},i}}(\text{AF_ID}, a_2)$. UE sends A-KID_{*i*} along with the ciphertexts $(\text{CT}_{\text{UE} \rightarrow \text{AF}}, \text{CT}_{\text{UE} \rightarrow \text{AAnF}})$ to AF in an Application_Session_Establishment_Request.
7. **Application Key Request:** AF forwards A-KID_{*i*}, its own identity (AF_ID), and the ciphertext $\text{CT}_{\text{UE} \rightarrow \text{AAnF}}$ to AAnF in Naanf_AKMA_ApplicationKey_Get_Request.
8. 9) **Avoid Sending SUPI and GPSI:** To reduce attack risks, AKMA+ does not share SUPI or GPSI with AAnF, preventing adversaries from linking the UE's identity.

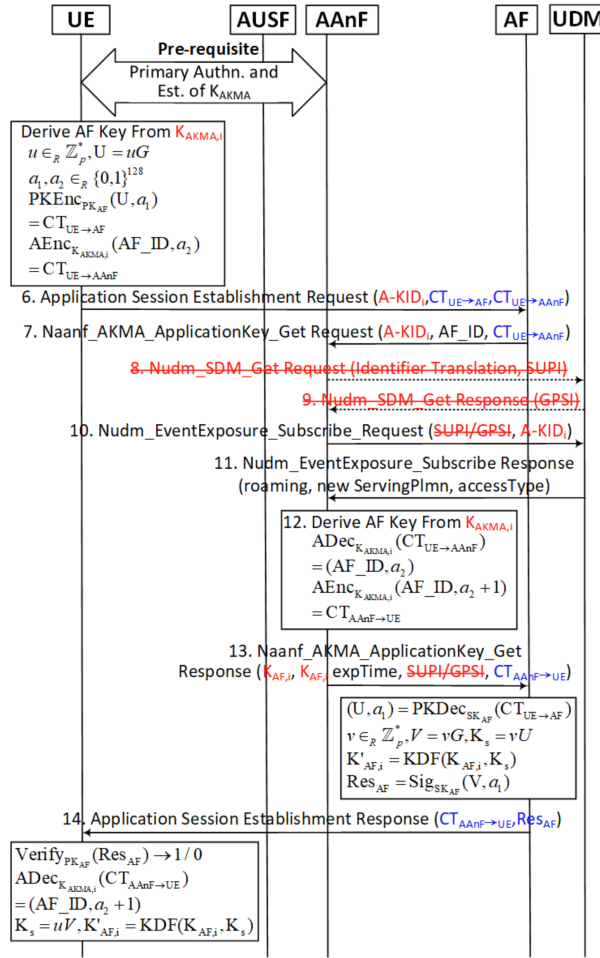


Fig. 6. AKMA+: Privacy-hardened Application Key Generation [14]

10. 11) **Roaming Status Check:** AAnF sends $A-KID_i$ to UDM to request a RoamingStatusReport. UDM retrieves the SUPI linked to $A-KID_i$ from its database and returns the UE's roaming status.
12. **Application Key Derivation by AAnF:** AAnF uses the same KDF as the UE to derive the application key $K_{AF,i}$ from $K_{AKMA,i}$.
13. **DH Key Exchange by AF:** Using $K_{AKMA,i}$ and the authenticated decryption algorithm $ADec$, AAnF decrypts $CT_{UE \rightarrow AAnF}$ to recover (AF_ID, a_2) . It verifies whether the recovered AF_ID matches the one sent by AF. If the verification fails, AAnF terminates the session. For mutual authentication, AAnF encrypts $(AF_ID, a_2 + 1)$ using $K_{AKMA,i}$, $CT_{AAnF \rightarrow UE} = AEnc_{K_{AKMA,i}}(AF_ID, a_2 + 1)$. AAnF sends the Naanf_AKMA_ApplicationKey_Get response to AF, including $K_{AF,i}$, the expiration time of $K_{AF,i}$, and $CT_{AAnF \rightarrow UE}$. Do not send SUPI or GPSI representing the

UE identity to the AF. AF decrypts $CT_{UE \rightarrow AF}$ using its secret key SK_{AF} to recover (U, a_1) . AF selects an ephemeral DH secret key $v \in_R \mathbb{Z}_p^*$ and calculates $V = vG$. A DH shared key is computed as $K_s = vU$, and a privacy-hardened application key $K'_{AF,i} = KDF(K_{AF,i}, K_s)$.

14. **Secure Communication:** AF generates Res_{AF} , which contains the signed hash result of (V, a_1) using its private key SK_{AF} , along with the plaintext of these two elements. AF sends $(CT_{AAnF \rightarrow UE}, Res_{AF})$ to UE in the Application Session Establishment Response. Upon receiving the response, UE verifies the signatures of (V, a_1) in Res_{AF} and utilizes $K_{AKMA,i}$ to recover $(AF_ID, a_2 + 1)$ from $CT_{AAnF \rightarrow UE}$. UE verifies whether the recovered nonces (a_1, a_2) match the original nonces. If the verification fails, UE terminates the session. Otherwise, UE computes $K_s = uV$, derives the privacy-hardened application key $K'_{AF,i}$ in the same way as AF, and establishes a secure channel with AF.

3.3 Key Differences Between AKMA and AKMA+

Table 1. Key Differences Between AKMA and AKMA+

Feature	AKMA	AKMA+
1.Key-Identifier Pairs	Generates a single key-identifier pair $(K_{AKMA}, A-KID)$ per UE	Generates multiple key-identifier pairs $\{K_{AKMA,i}, A-KID_i\}_{i=1,\dots,n}$ per UE and shuffles them among users
2.UAAnF Authentication	No direct mutual authentication between UE and UAAnF	Implements a challenge-response mechanism for mutual authentication using nonces
3.Application Key	Derives K_{AF} directly from K_{AKMA} ; No forward secrecy	Derives a privacy hardened application key $K'_{AF,i}$ using both $K_{AF,i}$ and a DH shared key
4.Session Freshness	No explicit mechanism for ensuring session freshness	Uses random nonces to ensure session freshness and resist replay attacks

1) Multiple Key-Identifier Pairs vs. Single Pair

AKMA uses a single key pair per user creates a privacy risk by allowing their activity to be tracked. AKMA+ solves this by generating and shuffling multiple key pairs among users, which prevents this linkage and protects user privacy.

2) Mutual Authentication Between UE and UAAnF

AKMA lacks direct mutual authentication, creating security risks from imposter UEs or UAAnFs. AKMA+ solves this by adding a nonce-based challenge-response mechanism, allowing the UE and UAAnF to securely verify each other's identity.

3) Privacy-Hardened Application Key

In AKMA, the AAnF can access the UE-AF session key, which also lacks forward secrecy. AKMA+ addresses this issue by introducing a DH key exchange between the UE and AF to generate a session key that is secret from the AAnF, thereby ensuring both confidentiality and forward secrecy.

4) Session Freshness and Replay Protection

AKMA is vulnerable to replay attacks because it lacks a session freshness mechanism. AKMA+ solves this by using random nonces to ensure every session is unique and cannot be replayed.

4 Attacks on AKMA+

Despite the advanced security features of AKMA+, our analysis shows a significant vulnerability arising from the compromise of the AAnF. This attack highlights that the protocol fails to fully achieve mutual authentication between the UE and AF.

4.1 AAnF Compromise Attack

The AKMA+ threat model explicitly considers AAnF as a covert entity that may act maliciously. The AKMA+ paper points out that since AAnF is outside the 5G core network and lacks the same security measures as those within the 5G core network, under the AKMA+ assumption that AAnF may be attacked by an adversary, we demonstrate that a compromised AAnF can leverage its privileged position to impersonate legitimate UEs to AFs.

Attack Prerequisites

- Adversary has compromised the AAnF and has access to stored AKMA anchor keys $K_{AKMA,i}$.
- The compromised AAnF continues to follow the protocol normally to avoid detection.

Attack Procedure

1. The compromised AAnF has access to multiple $\{K_{AKMA,i}, A-KID_i\}$ pairs stored for legitimate UEs.
2. For a targeted UE with anchor key $K_{AKMA,i}$, the adversary can derive the corresponding application key $K_{AF,i}$ using the same KDF as specified in Annex A.4 of TS 33.535.
3. The adversary initiates an impersonation attack by sending an Application_Session_Establishment_Request to the target AF with the following components:
 - A valid A-KID_i from the compromised AAnF
 - Ciphertext $CT_{UE \rightarrow AF} = PKEnc_{PK_{AF}}(U', a'_1)$ where U' and a'_1 are generated by the adversary
 - Ciphertext $CT_{UE \rightarrow AAnF} = AEnc_{K_{AKMA,i}}(AF_ID, a'_2)$ using the compromised $K_{AKMA,i}$, a'_2 generated by adversary.
4. When the AF forwards A-KID_i, AF_ID, and $CT_{UE \rightarrow AAnF}$ to the AAnF (which is controlled by the adversary), the adversary can:
 - Decrypt $CT_{UE \rightarrow AAnF}$ using $K_{AKMA,i}$
 - Verify that the recovered AF_ID matches the one sent by the AF

- Generate $CT_{AAnF \rightarrow UE} = AEnc_{K_{AKMA,i}}(AF_ID, a'_2 + 1)$
- Send $K_{AF,i}$ along with $CT_{AAnF \rightarrow UE}$ to the AF
- 5. The AF, unable to detect the impersonation, proceeds with the protocol:
 - Decrypts $CT_{UE \rightarrow AF}$ to recover (U', a'_1)
 - Generates its own DH component $V = vG$ and computes $K_s = vU'$
 - Derives the privacy-hardened application key $K'_{AF,i} = KDF(K_{AF,i}, K_s)$
 - Sends $(CT_{AAnF \rightarrow UE}, Res_{AF})$ to what it believes is the legitimate UE
- 6. The adversary, having initiated the session, can compute $K_s = u'V$ (where u' is the ephemeral DH secret used to generate U') and derive $K'_{AF,i} = KDF(K_{AF,i}, K_s)$.
- 7. The adversary now has established a secure channel with the AF while impersonating a legitimate user.

Attack Implications

This attack reveals that the AKMA+ protocol cannot achieve its security goal of mutual authentication between the UE and AF when the AAnF is compromised. The key findings are:

- The AF has no means to verify whether the session initiation genuinely originated from the legitimate UE or from an adversary with access to $K_{AKMA,i}$.
- The DH key exchange between the UE and AF, while providing forward secrecy, does not prevent impersonation attacks when the adversary has access to the long-term keys stored at the AAnF.
- The mutual authentication between UE and AAnF (Goal 4) does not extend to ensuring mutual authentication between UE and AF when AAnF is compromised.
- The privacy-hardened application key $K'_{AF,i}$ derived from both $K_{AF,i}$ and the DH shared key K_s does not provide additional security against this attack.

4.2 UE Compromise Attack

The AKMA+ document describes the long-term key K stored on the SIM card by the UE as secure and impervious to adversaries. However, other protocol-related information, such as $K_{AKMA,i}$ and $A-KID_i$, lacks the same protection mechanisms as the long-term key K and is therefore at risk of being compromised by adversaries. Based on the protocol description, we can determine that:

Storage of $K_{AKMA,i}$ in the UE

In the AKMA+ protocol, the AKMA anchor keys ($K_{AKMA,i}$) are derived during the primary authentication procedure between the UE and AUSF, specifically in Step 3 of the "Derive AKMA Anchor Keys" phase, where both the UE and AUSF generate $\{K_{AKMA,i}, A-KID_i\}_{i=1,\dots,n}$. The UE stores these keys for future authentication and key derivation, following the architecture described in Section 2.2. This architecture defines the UE as having two main components.

- Universal Integrated Circuit Card (UICC): This contains the Universal Subscriber Identification Module (USIM).
- Mobile Equipment (ME): This represents the physical device, such as a smartphone or IoT device.

AKMA+ protects the long-term key K in the secure UICC but stores the derived key $K_{AKMA,i}$ and identifier $A-KID_i$ on the ME. Since the ME is much more vulnerable to

malware, OS compromises, and physical attacks than the hardened USIM, this design significantly increases the risk of key theft.

Attack Procedure

This attack is identical to the AAnF Compromise Attack. The difference is that the adversary obtains $K_{AKMA,i}$ and $A-KID_i$ from the UE.

1. The adversary gains access to the locally stored $K_{AKMA,i}$ and $A-KID_i$ of a legitimate UE. This may occur due to device theft, malware, or other unauthorized access.
2. Using compromised $A-KID_i$, adversary initiates communication with an AF, presenting itself as a legitimate UE.
3. The adversary successfully derives $K_{AF,i}$ and establishes a secure session with AF.
4. The adversary's ability to impersonate the legitimate UE compromises the confidentiality, integrity, and authenticity of the communication between UE and AF. Additionally, the adversary may exploit this session to launch further attacks, such as data exfiltration or service disruption.

Attack Implications

The loss of $K_{AKMA,i}$ and $A-KID_i$ has far-reaching implications for the security of the AKMA+ protocol:

- The adversary can impersonate the legitimate UE, leading to potential tracking and profiling of user behavior.
- The adversary can hijack the communication session between UE and AF, gaining unauthorized access to sensitive application data.
- The mutual authentication mechanism between UE and AAnF is rendered ineffective, exposing the AKMA+ protocol to impersonation attacks.

Although the AKMA+ specification doesn't state where to store $K_{AKMA,i}$, its critical security role means it should not be kept in the ME's general memory. This is essential for the protocol's security. If implementation constraints force storage in the ME instead of the USIM, it would be a significant security compromise requiring additional safeguards, such as secure enclaves or trusted execution environments.

4.3 Ambiguity in Account Identity

Although AKMA+ enhances privacy and security over the standard AKMA protocol, it introduces a critical limitation: the absence of explicit user account identification mechanisms. This omission creates significant challenges for AFs in real-world deployments where user identification is essential for service delivery. As additionally stated in Clause 4.3.2 of TR 33.835, the AF must be capable of distinguishing among users; otherwise, it cannot deliver personalized services.

The fundamental problem with the AKMA+ protocol is its lack of a clear account identifier, which prevents applications from distinguishing between users. This inability to maintain user-specific data and preferences undermines the protocol's practical applicability for personalized services. Account-based identification is critical for modern applications. It enables service personalization by maintaining individual preferences and usage history, allowing services like Spotify to preserve user playlists and provide tailored recommendations. Furthermore, these systems are vital for data security and integrity, ensuring that information remains accessible only to authorized individuals

and maintaining clear data ownership boundaries. Finally, they guarantee service continuity; without persistent account identification, users would lose access to their saved data, purchased services, and personalized settings each time they reconnect to the service.

In critical applications like healthcare, reliable user identification is essential to prevent severe consequences such as medical errors and safety risks, which undermine system trust. While the AKMA+ protocol enhances privacy, its lack of an explicit identification mechanism creates a fundamental tension with practical service requirements. This limitation must be addressed to enable its safe and effective deployment in real-world scenarios where identification is crucial.

In the AKMA+ protocol, the AF only receives cryptographic identifiers that are not linked to a persistent user account. Consequently, the AF cannot identify which user is accessing the service, making it impossible to deliver personalized services, enforce access controls, or maintain session continuity. This design not only renders the protocol unusable in real-world applications requiring account-based services but also violates the 3GPP TR 33.835 requirement that AFs must be able to identify users.

Analysis of Account Name Integration Strategies

To ensure standard compatibility, the only feasible location for introducing an account identifier is within the sixth message sent from UE to AF in Figure 6. Below, we analyze four possible integration strategies and their associated risks:

1. **Adding a Separate Account Name Field Alongside Existing Fields:** If the protocol allows the UE to send a message containing $A\text{-}KID_i$, $CT_{UE \rightarrow AF}$, $CT_{UE \rightarrow AAnF}$, and a separate plaintext account name. The adversary obtains a valid $K_{AKMA,i}$ and $A\text{-}KID_i$ for themselves and learns the account name of a target user (e.g., via social engineering or prior compromise). The adversary constructs a message with their own $K_{AKMA,i}$ and $A\text{-}KID_i$, but inserts the target user's account name in the account name field and sends this message to the AF.
The AF, unable to cryptographically bind the account name to the key material, accepts the adversary's request as if it were from the target user, effectively allowing account takeover. This enables impersonation attacks, violating both authentication and privacy guarantees.
2. **Embedding the Account Name in $CT_{UE \rightarrow AF}$:** If the account name is included in $CT_{UE \rightarrow AF}$, which is encrypted under AF's public key. The adversary generates their own ephemeral key pair and $K_{AKMA,i}$ and $A\text{-}KID_i$, chooses a victim's account name and embeds it within the plaintext before encryption. The adversary encrypts the message with AF's public key, producing a valid $CT_{UE \rightarrow AF}$ and sends the forged message to the AF.
AF cannot distinguish whether the account name originated from the legitimate UE or an attacker, thus enabling unauthorized access to the victim's account.
3. **Embedding the Account Name in $CT_{UE \rightarrow AAnF}$:** If the UE includes the account name in the plaintext before encrypting with $K_{AKMA,i}$. The resulting ciphertext $CT_{UE \rightarrow AAnF}$ is sent to AF and then relayed to AAnF. However, AF does not possess $K_{AKMA,i}$ and cannot decrypt $CT_{UE \rightarrow AAnF}$. AF is unable to extract or verify the account name, rendering the field useless for user identification, also fails to meet the requirement for AF to identify users.

4. Including the Account Name in A-KID_i: Two sub-cases:

- *Encrypted Account Name*: If the account name is encrypted, AF must be able to decrypt. The only practical method is to use the AF's public key to encrypt account name and add the ciphertext to the A-KID_i, which brings us back to the vulnerabilities described in case 2.
- *Plaintext Account Name*: If the account name is included in plaintext, to prevent adversaries from inserting arbitrary account names, AAnF must maintain a (SUPI, AF ID, Account Name) mapping for all users and AFs. However, since A-KID_i is transmitted in plaintext, both passive adversaries and AAnF can observe every login attempt associated with a specific account.

The structural limitations inherent in the current AKMA+ framework make it infeasible to simply introduce usernames as a solution. Specifically, the protocol lacks explicit fields for user account identifiers, which renders any attempt to incorporate usernames incompatible with the existing cryptographic elements, such as A-KID_i and ciphertexts (e.g., $CT_{UE \rightarrow AF}$ and $CT_{UE \rightarrow AAnF}$). This design flaw means that the protocol cannot ensure strong consistency between usernames and user 5G identity (SUPI), resulting in the introduction of usernames that cannot be reliably verified or securely transmitted. Furthermore, the encryption mechanisms and message exchange logic within the current framework do not accommodate the processing or transmission of usernames, leaving no room for their integration without disrupting the protocol's original structure. As a result, the limitations of the current framework preclude the possibility of addressing the ambiguity in user account identification through the simple inclusion of usernames. Instead, a fundamental redesign of the protocol is required to overcome these structural deficiencies.

Privacy Issue

Embedding the account name in plaintext into A-KID_i is the only practical solution, but it creates severe privacy and security risks. It allows the AAnF to monitor all user logins, and external adversaries can track activities by eavesdropping on the plaintext A-KID_i. This undermines user anonymity and poses significant risks of profiling and tracking.

This tracking vulnerability poses significant risks to user privacy and creates serious compliance issues with the General Data Protection Regulation (GDPR) [12]. GDPR mandates that personal data, including identifiers, be protected from unauthorized access and profiling, requiring robust safeguards by design. The ability for an AAnF or external adversaries to track accounts directly violates several key GDPR principles: Article 5 (data minimization) is breached by exposing unnecessary identifiers; Article 6 (lawfulness) is violated by tracking without consent; and Article 25 (data protection by design) is ignored, as the AKMA+ protocol lacks adequate privacy measures. These vulnerabilities expose users to profiling and tracking, and providers to legal and financial penalties.

5 Architectural Problems in AKMA+: Root Cause Analysis

5.1 Fundamental Trust Assumptions in AKMA+

The AKMA+ protocol, like its predecessor AKMA, is built upon a new centralized trust model where the AAnF serves as the primary trust anchor. This architectural decision creates an inherent security bottleneck:

1. **Single Point of Trust:** The AAnF acts as the sole mediator between UE and AFs, responsible for key distribution and authentication verification.
2. **Unilateral Trust Dependency:** UEs can only prove their identity to AFs through credentials validated by the AAnF, creating a unidirectional trust relationship.
3. **Concentrated Cryptographic Material:** The AAnF stores AKMA anchor keys ($K_{AKMA,i}$) for all users, creating a high-value target for attackers.

This centralized architecture was designed to simplify key management and leverage the existing 3GPP authentication infrastructure. However, our analysis demonstrates that this design creates inherent vulnerabilities that cannot be fully mitigated without architectural changes.

5.2 Root Cause Analysis of Identified Vulnerabilities

1) AAnF Compromise Attack

The vulnerability enabling user impersonation when the AAnF is compromised stems directly from the architectural decision to make the AAnF the sole trust anchor in the system. When the entity responsible for validating user identity is compromised, the entire authentication chain collapses. This is not merely an implementation issue but a fundamental architectural limitation:

- The UE's trust root for providing identity authentication comes exclusively from the AAnF.
- No independent verification path exists between UE and AF that doesn't rely on the AAnF.
- The AF has no means to verify UE authenticity independent of the AAnF's assertions.

Under the current architecture, this vulnerability cannot be effectively addressed through standard-compatible protocol optimizations alone; achieving robust security requires a fundamental reconsideration of the trust model.

2) UE Compromise Attack

The transition from ephemeral keys in AKMA to persistent keys in AKMA+ introduces architectural challenges related to key management:

- The architecture assumes secure, long-term storage capabilities for cryptographic material across multiple entities.
- Key revocation and rotation mechanisms become more complex with persistent keys.
- The expanded attack surface due to longer key lifetimes is an inherent consequence of the architectural decision to support privacy through multiple persistent keys.

While this vulnerability is partially addressable through implementation-level security enhancements (such as hardware security modules), the fundamental tension between privacy requirements and security exposure remains an architectural challenge.

3) Ambiguous about Account Identity

The architectural limitations in AKMA+ regarding account identity and privacy protection stem from fundamental protocol design choices:

- The protocol does not natively define an account identity field. Attempts to add account identifiers—whether in plaintext or encrypted form—lead to risks.
- Embedding account information directly exposes sensitive user data and may violate privacy regulations.
- The protocol structure cannot reconcile the conflict between identity recognition and privacy protection; maybe a fundamental redesign can resolve these architectural flaws.

The AKMA+ specification is ambiguous on this point, and the current architecture makes it difficult to balance privacy with anonymity. Resolving this conflict will likely require significant changes to the protocol's design.

5.3 Architectural Limitations in Untrusted AAnF Scenarios

The most critical architectural limitation of AKMA+ becomes apparent when considering scenarios where the AAnF cannot be fully trusted, a risk that the authors of the AKMA+ paper themselves acknowledge. As stated in Section 3.3 of the paper, the AAnF is considered a "covert entity" that may act maliciously under certain conditions.

1. **Lack of Independent Trust Paths:** The current architecture provides no alternative trust path for UEs to authenticate to AFs when the AAnF is compromised or malicious.
2. **Absence of Trust Verification:** AFs have no mechanism to verify that the AAnF is behaving correctly or to detect if it has been compromised.
3. **Concentrated Authority:** The AAnF has unilateral authority to issue application keys, with no checks and balances in the architecture.
4. **Privacy-Security Tradeoff:** While multiple keys enhance privacy, the need to store them increases security risks.

These limitations highlight that the current AKMA+ architecture is fundamentally built on the assumption that the AAnF is trustworthy. When this assumption is violated, the security and privacy guarantees of the protocol collapse.

5.4 Potential Enhancements for AKMA+

Despite the advancements introduced by AKMA+, our analysis indicates that fundamental architectural limitations persist, particularly regarding centralized trust and persistent key storage. To address these challenges and further strengthen the security and privacy of 5G authentication frameworks, we propose several potential enhancement directions for AKMA+.

1) Trusted Execution Environment Leverage

Utilizing hardware-based security features can enhance protection even in potentially compromised environments. For example, storing and processing cryptographic

keys within Trusted Execution Environments (TEEs) helps protect against software-based attacks [7]. This approach significantly raises the difficulty for attackers, as successful compromises would require breaching hardware-level defenses rather than relying solely on software vulnerabilities, thereby enhancing overall system security. While this approach introduces manageable performance overhead and development complexity, it is a justified trade-off for preventing UE impersonation attacks by significantly raising the bar for key theft. Crucially, this enhancement maintains full 3GPP compatibility, as it fortifies the UE's internal security without altering the external protocol messages, interfaces, or the overall AKMA+ architecture

2) Change the Protocol Architecture.

By modifying the protocol architecture and incorporating the user's account name into key information during the authentication process, the AF can leverage this key information to identify the UE's account and deliver personalized services. For instance, a legitimate UE can associate its username with a pair of public and private keys, effectively preventing malicious UEs from impersonating it.

6 Conclusion

This paper presents a comprehensive security and privacy analysis of the AKMA+ protocol, an enhanced version of the AKMA framework for 5G networks. While AKMA+ introduces significant improvements, such as UE indistinguishability mechanisms to address privacy concerns, our analysis identifies three critical vulnerabilities. These vulnerabilities stem from the fundamental architectural limitations of AKMA+, particularly its centralized trust model, which inherently weakens security and privacy guarantees.

The AKMA+ protocol represents progress in addressing the security and privacy challenges associated with 5G networks. Our analysis indicates current protocol designs fail to fully comply with 3GPP requirements, especially when critical components like AAnF are compromised. Specifically, the existing architecture struggles to effectively mitigate attacks stemming from the compromise of the AAnF, as within this framework, the AF relies entirely on the services provided by the AAnF for user authentication.

To address this issue, we propose several protocol enhancement directions, including hardware-based secure storage [7] and changing the protocol architecture. However, these countermeasures may require significant modifications to the existing AKMA architecture. This highlights the need for further research and development to design an AKMA framework that is both fully secure and capable of ensuring robust privacy protection.

References

1. 3GPP: Nr; nr and ng-ran overall description; Tech. Rep. TS 38.300, 3rd Generation Partnership Project (2018), <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3191>

2. 3GPP: Study on authentication and key management for applications based on 3gpp credential in 5g. Tech. rep., Technical Report TR 33.835 (July 2020)
3. 3GPP: Study on authentication and key management for applications based on 3gpp credential in 5g. Tech. Rep. TR 33.835, 3rd Generation Partnership Project (2020), <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3483>
4. 3GPP: Authentication and key management for applications (akma) based on 3gpp credentials in the 5g system (5gs). Tech. rep., Technical Specification TS 33.535 (July 2024)
5. 3GPP: Authentication and key management for applications (akma) based on 3gpp credentials in the 5g system (5gs). Tech. Rep. TS 33.535, 3rd Generation Partnership Project (2025), <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3690>
6. Akman, G., Ginzboorg, P., Damir, M.T., Niemi, V.: Privacy-Enhanced AKMA for Multi-Access Edge Computing Mobility. *Computers* **12**(1), 2 (2022)
7. Cerdeira, D., Santos, N., Fonseca, P., Pinto, S.: Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 1416–1432. IEEE (2020)
8. Ericsson: 5g technology by ericsson (2025), <https://www.ericsson.com/en/5g>, accessed: 2025-07-29
9. (ETSI), E.T.S.I.: 5g technology overview (2025), <https://www.etsi.org/technologies/mobile/5g>
10. GSMA: The 5g era: How 5g is changing the world (2025), <https://www.gsma.com/solutions-and-impact/technologies/networks/the-5g-era-how-5g-is-changing-the-world/>
11. Khan, M., Ginzboorg, P., Niemi, V.: Privacy preserving akma in 5g. In: Proceedings of the 5th ACM Workshop on Security Standardisation Research Workshop. pp. 45–56. ACM (2019)
12. Voigt, P., Von dem Bussche, A.: EU General Data Protection Regulation (GDPR). A Practical Guide, 1st Ed., Cham: Springer International Publishing **10**(3152676), 10–5555 (2017)
13. Yang, T., Wang, S., Zhan, B., Zhan, N., Li, J., Xiang, S., Xiang, Z., Mao, B.: Formal Analysis of 5G Authentication and Key Management for Applications (AKMA). *Journal of Systems Architecture* **126**, 102478 (2022)
14. Yang, Y., Yang, G., Li, Y., Huang, M., Shen, Z., Karim, I., Sasse, R., Basin, D., Bertino, E., Weng, J., et al.: AKMA+: Security and Privacy-Enhanced and Standard-Compatible AKMA for 5G Communication. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 5327–5345 (2025)